

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Tue 4 FEBRUARY 2020 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

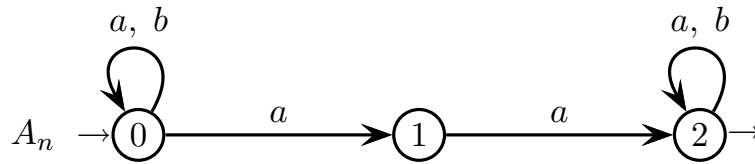
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the nondeterministic automaton A_n below, with a two-letter input alphabet $\Sigma = \{ a, b \}$:



Answer the following questions:

- (a) By using the the Berry-Sethi (*BS*) method, obtain a deterministic automaton A_d equivalent to automaton A_n .
 - (b) Transform the deterministic automaton A_d into a right-unilinear grammar G .
 - (c) Transform the above grammar G into a system of language equations and solve the system, so as to obtain a regular expression R_1 for the language accepted by automaton A_d .
 - (d) By using the Brzowski-McClyiskey (*BMC*) method (node elimination method), obtain a regular expression R_2 from automaton A_d .
 - (e) (optional) Determine whether the regular expressions R_1 and R_2 obtained at the previous points (c) and (d) are ambiguous. Justify your answer.
-

2 Free Grammars and Pushdown Automata 20%

1. Consider the Dyck language L_{Dyck} with two pairs of matching open / closed parentheses, i.e., a / b and c / d . A well-known non-ambiguous grammar G , over the terminal alphabet $\Sigma = \{ a, b, c, d \}$, that generates language L_{Dyck} is shown below (axiom S):

$$G \left\{ \begin{array}{l} S \rightarrow a S b S \\ S \rightarrow c S d S \\ S \rightarrow \varepsilon \end{array} \right.$$

Answer the following questions:

- (a) Write two *BNF* grammars G_1 and G_2 (no matter if ambiguous), respectively for the languages $L_1, L_2 \subset L_{Dyck}$ defined as follows:

- (i) L_1 contains all the strings of L_{Dyck} except those that have the digram ac
- (ii) L_2 contains all the strings of L_{Dyck} except those that have the digram dc

Suitably modify grammar G , but do not use more than three nonterminals for each grammar. Verify that the following is true (draw the trees when possible):

- (i) $acdb \notin L_1$ and $cabd \in L_1$
- (ii) $cdcd \notin L_2$ and $cdab \in L_2$

- (b) Now consider the regular expression R below:

$$R = (a \mid b)^* (c \mid d)^*$$

Write a non-ambiguous *BNF* grammar G_3 that generates the language $L_3 \subset L_{Dyck}$ below:

$$L_3 = L_{Dyck} \cap L(R)$$

Suitably modify grammar G , but do not use more than three nonterminals. Reasonably argue that grammar G_3 is not ambiguous.

- (c) (optional) Consider the well-known *EBNF* grammar below (axiom S), which generates the Dyck language with only one parenthesis pair, i.e., a / b :

$$S \rightarrow (a S b)^*$$

Suitably modify this grammar, extend the terminal alphabet to $\Sigma = \{ a, b, c, d \}$, and write two *EBNF* grammars G^E and G_3^E that have the following properties:

- (i) grammar G^E is weakly equivalent to grammar G
- (ii) grammar G_3^E is weakly equivalent to grammar G_3

For both grammars G^E and G_3^E , do not use more than *one* nonterminal, i.e., S .

2. A programming language fragment includes variables and constants, relational and boolean expressions, assignment and conditional statements, specified as follows:

- A conditional statement, within the keywords “if” and “fi”, orderly contains:
 - a condition, which consists of any expression
 - a clause *then*, with keyword “then” and a non-empty list of statements
 - zero, one or more clauses *elsif*, with keyword “elsif”; each such clause contains a condition and only one clause *then*, which is as specified before
 - an optional clause *else*, with keyword “else” and a non-empty list of statements
- An expression contains the following operators: equality “=”, negation “not”, n -ary conjunction “and” and n -ary disjunction “or” (both with $n \geq 2$), and implication “imp” (only binary). As usual, round parentheses may enclose subexpressions and allow the user to override the default operator precedence.
- The precedence of the operators, in decreasing order from the highest, i.e., maximal priority, to the lowest, i.e., minimal priority, is the following: equality, negation, conjunction, disjunction and implication.
- An assignment statement, with the operator “:=”, assigns an expression (on the right-hand side) to a variable (on the left-hand side).
- The lexical tokens for the identifiers are schematized by terminal id, which does not need to be further expanded.
- A phrase of the language is a non-empty list of assignment and conditional statements, each of them terminated by a semicolon.

Here is a sample phrase of the language, from which you should infer any detail not included in the above specifications of the language syntax:

```
a := b;
if not a = c and b = c imp e and f and g and ( d or h )
  then
    c := e;
    e := f and g;
  elsif not e
    then
      if ( c = a or f ) and g
        then b := a;
        else b := e;
      fi;
    e := d;
  fi;
```

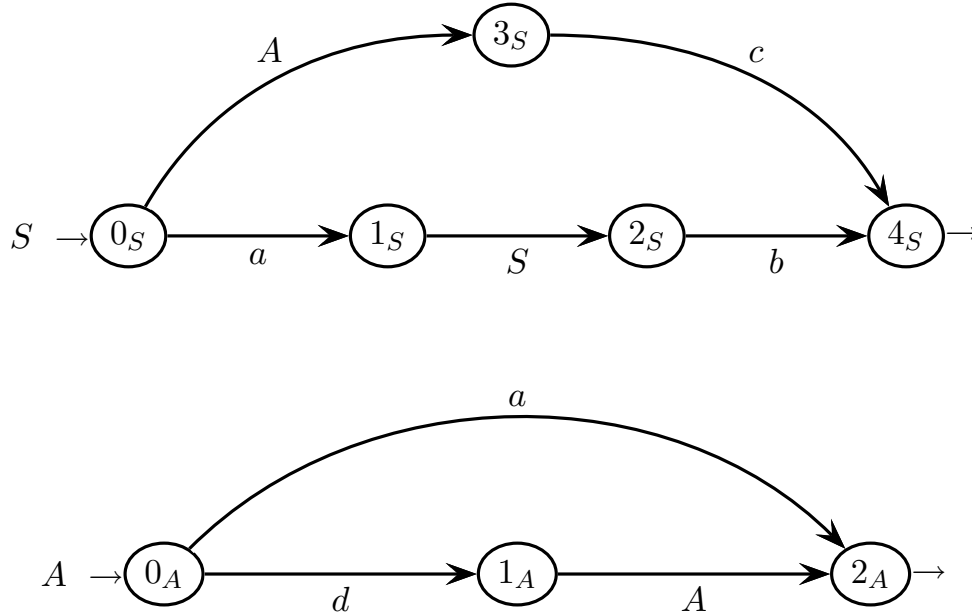
Answer the following questions:

- (a) Write a grammar, of type *EBNF* and not ambiguous, that generates the language sketched above.
- (b) To test the correctness of your grammar, draw the syntax subtree of the expression at line two of the sample phrase, i.e., the subtree of the expression:

```
not a = c and b = c imp e and f and g and ( d or h )
```


3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, over the four-letter terminal alphabet $\Sigma = \{ a, b, c, d \}$ and the two-letter nonterminal alphabet $V = \{ A, S \}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Draw the complete pilot automaton of grammar G . Show that grammar G is of type $ELR(1)$. Justify your answer.
- (b) Find all the guide sets on the call arcs and exit arrows of the $PCFG$ of grammar G . Show that grammar G is not of type $ELL(1)$. Justify your answer.
- (c) Simulate the ELR recognition of the sample valid string below:

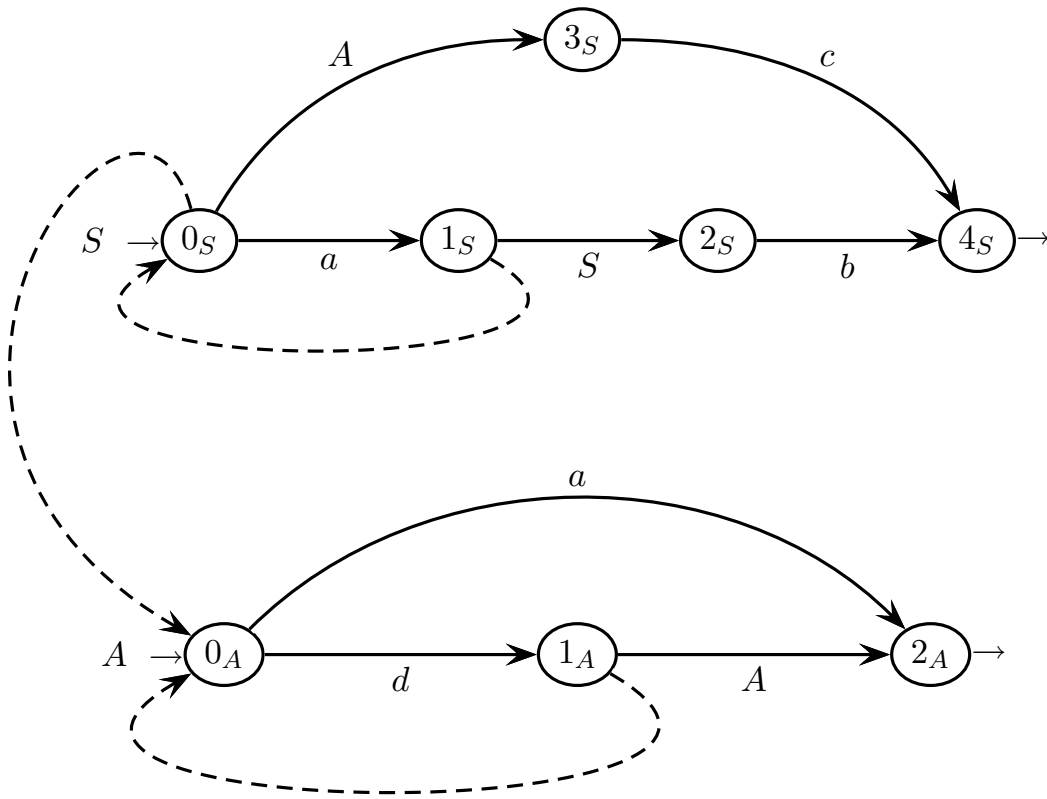
$$a a c b \in L(G)$$

and build the corresponding syntax tree. Show the correspondence between each subtree and each reduction move in the simulation.

- (d) (optional) Determine whether grammar G is $ELL(k)$ for some $k \geq 2$. Justify your answer.

please draw here the complete pilot automaton – question (a)

please write here the guide sets (on the call arcs and exit arrows) – question (b)



simulation table for the *ELR* analysis to be completed - question (c) - (the number of rows is not significant)

<i>stack of alternated macro-states and grammar symbols</i>	<i>input left to be scanned</i>	<i>move executed</i>
I_0	$a\ a\ c\ b\ \dashv$	initial configuration
$I_0\ a\ I_{\dots}$	$a\ c\ b\ \dashv$	terminal shift: $I_0 \xrightarrow{a} I_{\dots}$

draw here the piecewise tree construction

space for answering question (d)

free space for correction or continuation – any question

4 Language Translation and Semantic Analysis 20%

1. The grammar G_s below generates the source language L_s of the bit strings of even length ≥ 2 , over the binary source alphabet $\Sigma = \{ 0, 1 \}$ (axiom S):

$$G_s \left\{ \begin{array}{l} S \rightarrow 0 Q \mid 1 R \\ Q \rightarrow 0 S \mid 1 S \mid 0 \mid 1 \\ R \rightarrow 0 S \mid 1 S \mid 0 \mid 1 \end{array} \right.$$

We intend to define and compute a translation τ that maps the strings of language L_s onto the strings in base four, hence strings over the four-digit destination alphabet $\Delta = \{ 0, 1, 2, 3 \}$, that have the same numerical value.

For instance the binary string 10011100, which has decimal value 156, is mapped by translation τ to the base-four string 2130, which has the same decimal value:

$$\tau(10011100) = 2130 \quad \text{-- both strings have decimal value 156}$$

Answer the following questions (use the tables / drawings / spaces on the next pages):

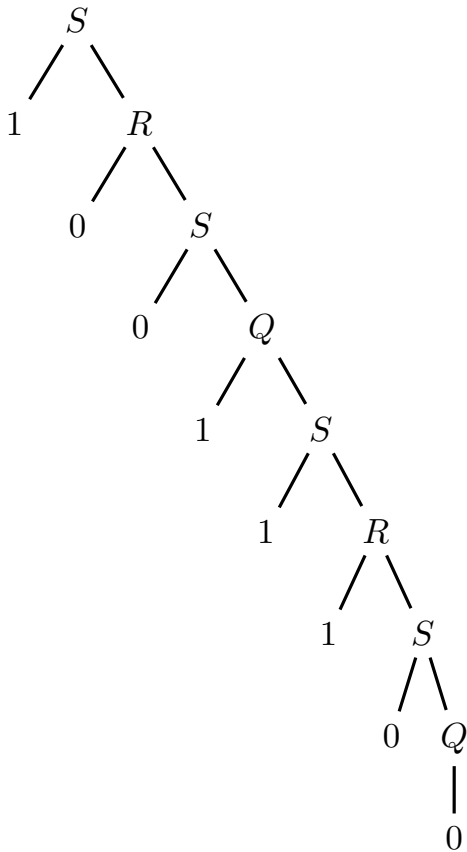
- (a) Write the destination grammar G_d (or the translation grammar G_τ) that defines the translation τ described above.
- (b) Draw the destination (or translation) syntax tree for the sample source string:
10011100
- (c) Write a regular translation expression (rational expression) R_τ that defines the above translation τ and design a finite-state transducer A_τ that computes it.
- (d) (optional) Write the recursive descent *ELL*-based procedures, only for the non-terminals S and R , that read the source string, compute translation τ and output the destination string.

please write here the destination (or translation) grammar – question (a)

$$G_s \left\{ \begin{array}{l} \overline{S \rightarrow 0 Q \mid 1 R} \\ \overline{Q \rightarrow 0 S \mid 1 S \mid 0 \mid 1} \\ \overline{R \rightarrow 0 S \mid 1 S \mid 0 \mid 1} \end{array} \right. \quad G_d \text{ or } G_\tau \left\{ \begin{array}{l} \overline{S \rightarrow} \\ \overline{Q \rightarrow} \\ \overline{R \rightarrow} \end{array} \right.$$

source tree

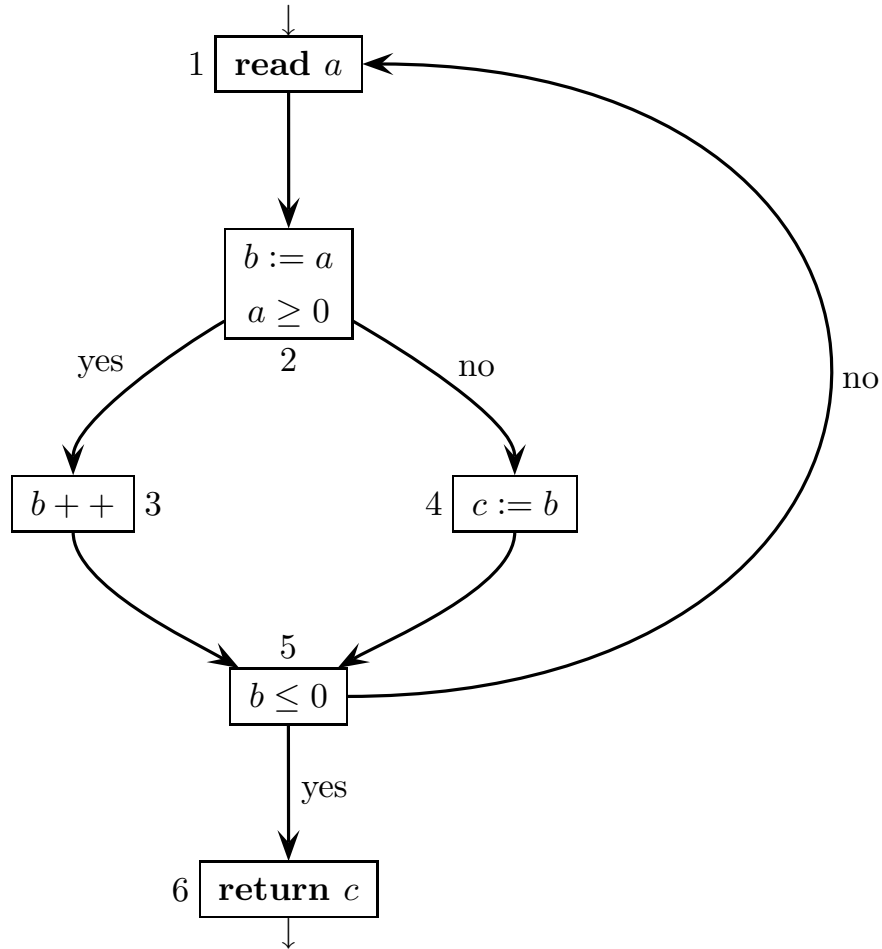
please draw here the destination
(or translation) tree – question (b)



space for answering questions (c-d) – continue elsewhere if necessary

free space for correction or continuation – any question

2. Consider the program Control Flow Graph (*CFG*) shown below, with six nodes:



The program has three integer variables: a , b and c .

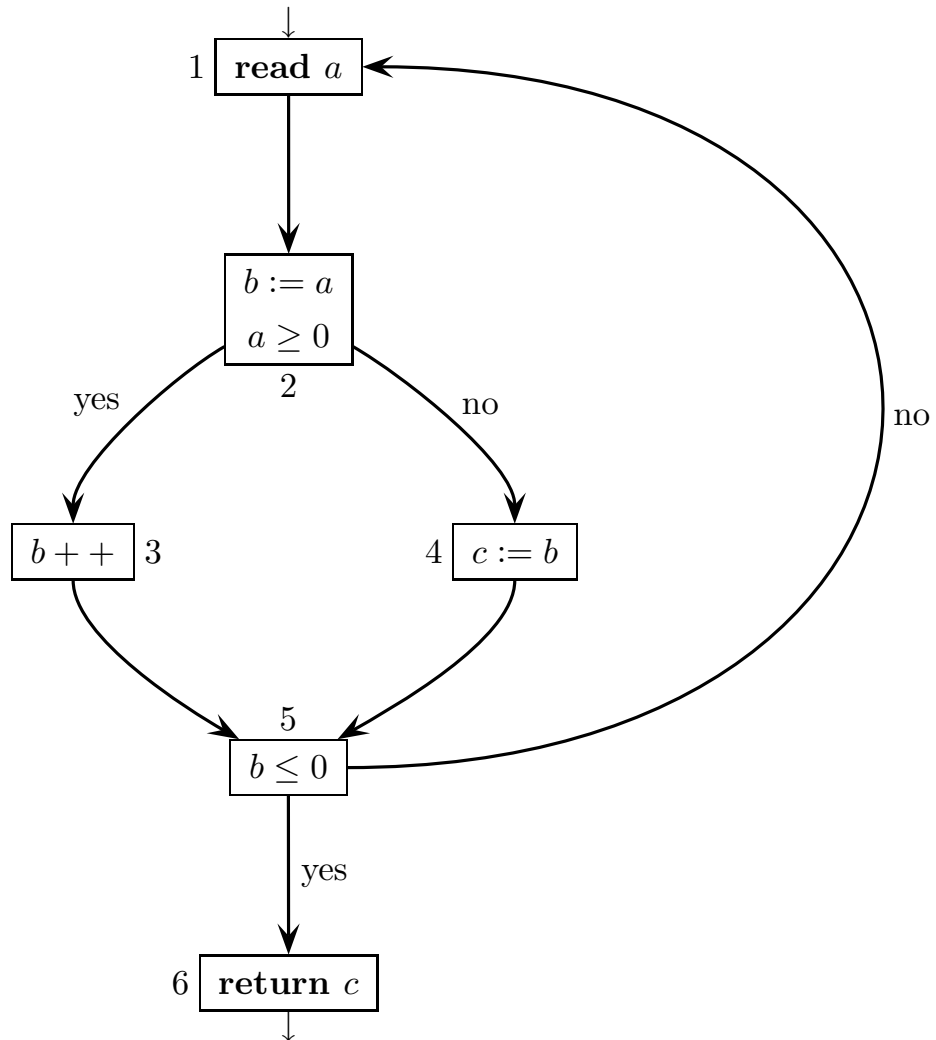
Answer the following questions (use the tables / drawings / spaces on the next pages):

- Disregarding the program semantics, write a regular expression R that represents the execution traces of the *CFG* over the node name alphabet $\{ 1, \dots, 6 \}$.
- Informally find the live variables (on the node input) at each node and write them by each node. Can the found liveness intervals be exploited to optimize the memory allocation for the program? Justify your answer.
- Find the variables defined and used at each node. Then write the flow equations (*in* and *out*) of the live variables, but only for the nodes 2, 3 and 5 (do not solve such equations). Verify that the solution found at point (b) satisfies the flow equations of nodes 2, 3 and 5.
- (optional) List all the variable definitions. Then informally find the reaching definitions (on the node output) at each node and write them by each node. If we consider the program semantics, what changes about the reaching definitions at the input node 1 and at the output node 6?

please write here the regular expression R – question (a)

$R =$

please write here the *live variables* at the *node input* – question (b)



space for answering question (c)

tables of definitions and usages at the nodes

<i>node</i>	<i>defined</i>	<i>node</i>	<i>used</i>
1		1	
2		2	
3		3	
4		4	
5		5	
6		6	

(partial) system of data-flow equations

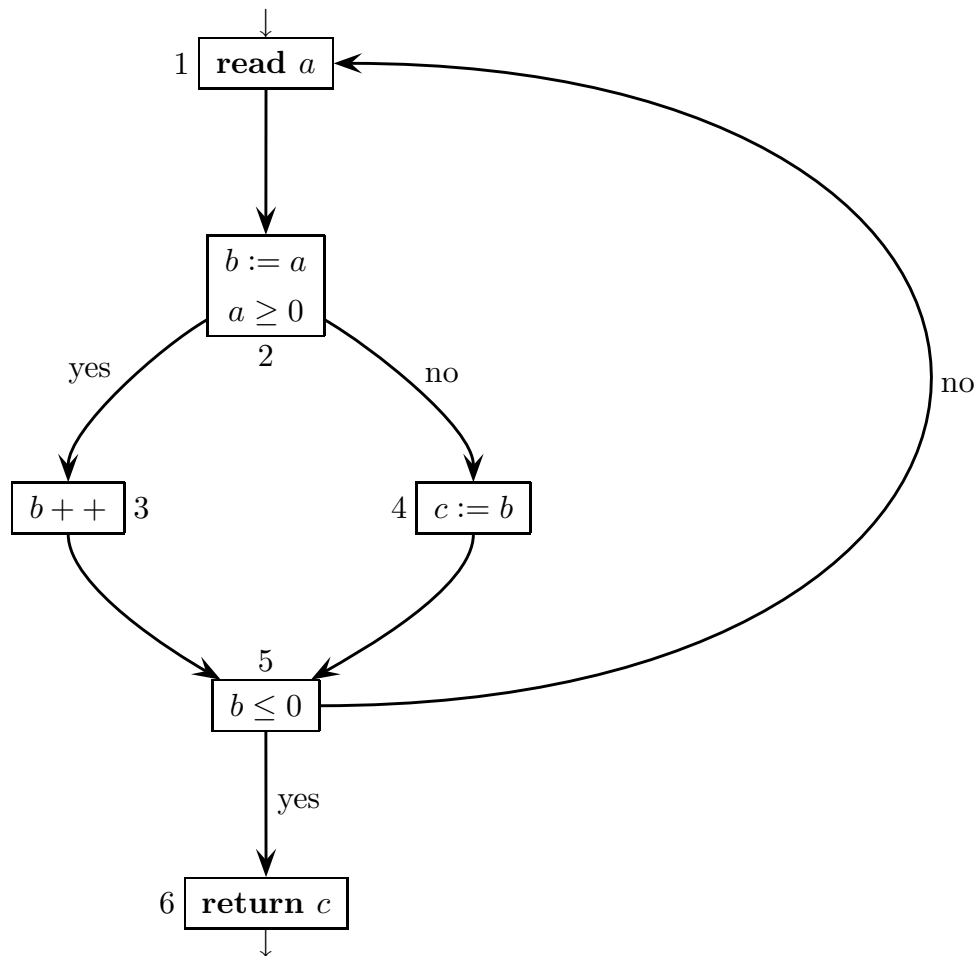
<i>node</i>	<i>in equations</i>	<i>out equations</i>
2		
3		
5		

space for the equation verification

space for answering question (d)

please write here the *list of variable definitions*

please write here the *reaching definitions* at the node output



please (shortly) describe here the changes caused by program semantics

free space for correction or continuation – any question